

NETRAJYOTI AI

High-Level Design (HLD) | Current Capstone Implementation and Future Controlled Scope

Scope statement

NetraJyoti is a Bengali-first deterministic eye-care guidance MVP. PostgreSQL is both the criteria repository for routing and the RAG knowledge store for route guidance. Java fixes the route; local Ollama can only explain that fixed route after Java validation. It is a capstone demonstration, not a clinical decision-support product.

1. Architecture overview and current implementation

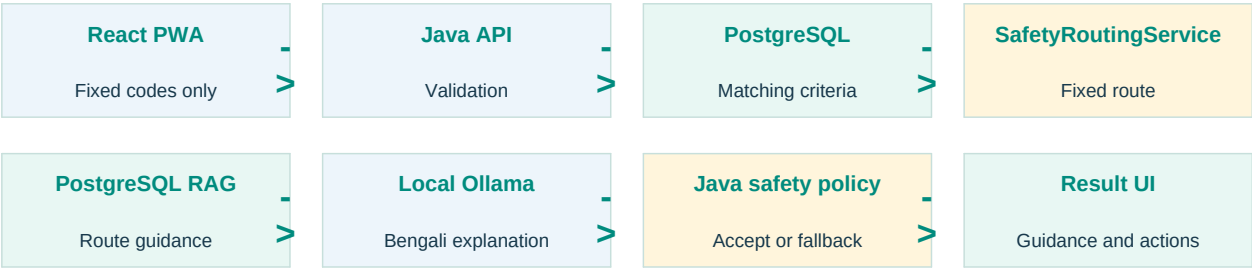


Diagram reading guide

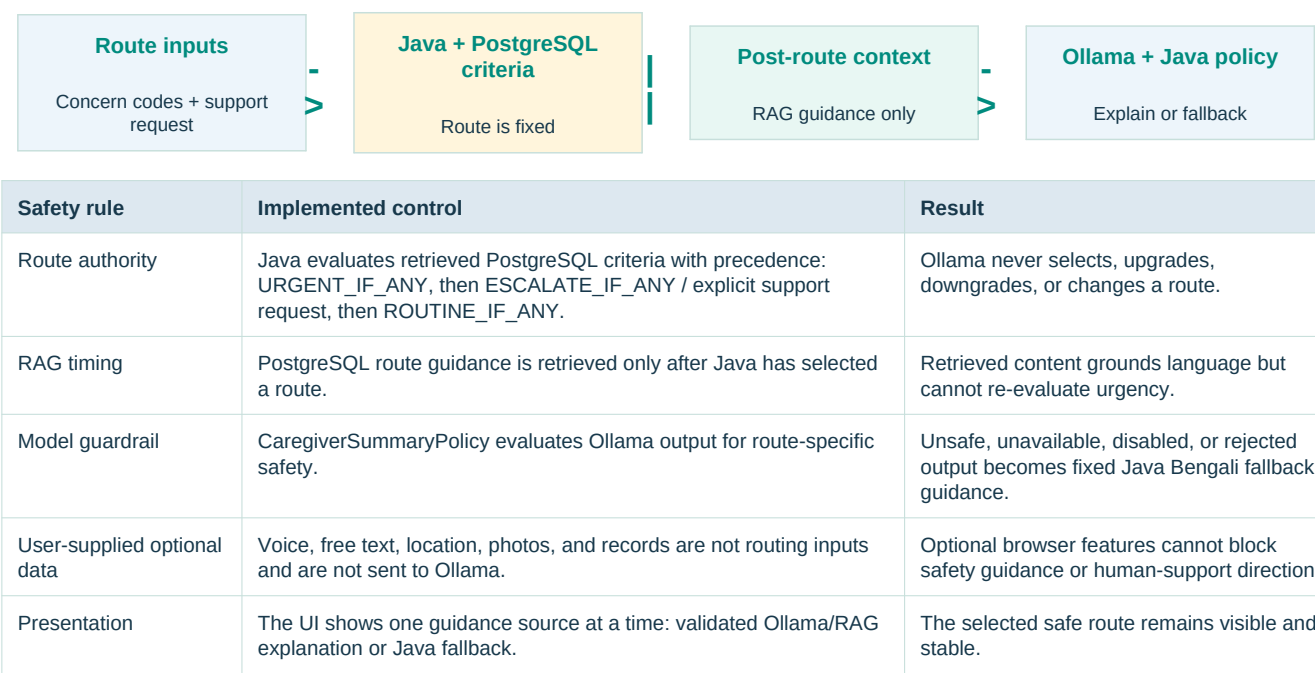
The upper row is the safety-critical route decision. The lower row is post-route communication. The double bar in the safety-boundary diagram means that information beyond it must not change the route.

Current MVP logical architecture

Layer	Component	Current responsibility
Experience	React + TypeScript PWA	Guided screening, local workflow state, selected symptoms, service direction, share/copy, and optional browser capabilities.
Application	Java 21 + Spring Boot REST API	Validates structured input, applies CORS/security configuration, coordinates routing, feature flags, Ollama, validation, and fallback.
Safety domain	SafetyRoutingService	Retrieves matching PostgreSQL criteria and applies a fixed, reviewable route precedence.
Data	PostgreSQL + Spring JDBC/JPA	Stores pathways, criteria, Bengali guidance, and reference sources. Current seed records are DEMO_SIMULATED_NOT_FOR_CLINICAL_USE.
AI explanation	Local Ollama (optional)	Creates a short Bengali explanation from route-specific PostgreSQL guidance only after Java has fixed the route.
Care navigation	Frontend demo options	Shows demo PHC, eye-care, ASHA, and family-support direction. It is not a verified managed directory.

2. Current safety boundary

Only fixed concern codes, an explicit human-support request, and PostgreSQL criteria are allowed to affect routing. History on the optional Step 3A screen is guidance context in the current demo and does not override the route.



Demo-data boundary

Seeded pathways, criteria, Bengali guidance, and sources are labelled DEMO_SIMULATED_NOT_FOR_CLINICAL_USE and require DEMO_RAG_ENABLED=true. They demonstrate the flow but are not approved clinical content or a production release.

3. PostgreSQL RAG and local Ollama

Stage	Current behaviour	Safety boundary
Feature flags	AI_SUMMARY_ENABLED enables local Ollama; DEMO_RAG_ENABLED enables simulated demo records.	Both are backend configuration.
RAG retrieval	PostgreSQL returns Bengali guidance and reference data for the fixed route.	No RAG record can alter the Java route.
Generation	Local Ollama (default llama3.2:3b) produces short Bengali communication.	No raw voice, free text, patient details, precise location, photos, or records are sent.
Validation	CaregiverSummaryPolicy accepts safe text or rejects it.	The API uses fixed fallback guidance on rejection or failure.
Presentation	The Result UI shows validated generated guidance or the Java fallback.	Ollama cannot diagnose, prescribe, invent facilities, or take autonomous action.

4. Security, privacy, and governance controls

Control area	Implemented now	Current boundary / required production strengthening
API exposure	Spring Security is stateless; CORS permits only the configured origin. Routing, AI-summary, and health endpoints are public for the demo.	This is suitable only for a local capstone demo. Production needs authenticated roles, HTTPS, rate limiting, abuse protection, and endpoint review.
Secrets and configuration	Database URL, username, password, CORS origin, Ollama URL, model, and feature flags are environment-based.	Use a managed secret store, rotation, least-privilege database credentials, and deployment configuration controls.
Data minimisation	Java receives structured concern/history codes and a boolean support request. Optional free text and browser features do not drive routing or go to Ollama.	Define data inventory, consent/notice, retention, deletion, access review, and incident response before handling real users.
Content governance	Database records have statuses and the demo switch makes simulated records explicit.	A clinical reviewer workflow, approval evidence, versioning, publication controls, and rollback are not implemented.
AI governance	Java route authority, post-route RAG, output policy, and fallback limit model impact.	Add Bengali safety evaluation, prompt/version tracking, red-team testing, performance monitoring, and formal clinical review.
Audit and operations	Backend logs routing inputs, matched criteria, selected outcome, generation, validation, and fallback reason.	Persistent non-identifying audit events, dashboards, alerts, retention policy, and formal incident handling are future scope.

5. Current capstone gaps and roadmap

Gap	Why it matters	Recommended next control
No authenticated administrator roles	Clinical reviewer, content administrator, operations staff, and facility staff are not separated.	Introduce identity, RBAC, approval queues, and activity history.
Demo clinical content	Seeded data is explicitly simulated and not for clinical use.	Clinical review, evidence sources, approved versions, release sign-off, and rollback.
Demo service directory	Facility/support options are frontend demonstration data without freshness verification.	Partner-owned directory, verified source, expiry/freshness metadata, and manual override.
No durable audit store	Logs alone are insufficient for governance or operational review.	Store non-identifying event records with correlation ID, configuration/content version, latency, validation, and fallback outcome.
Local development deployment	Backend runs locally; Docker Compose covers PostgreSQL and Ollama only.	Secure hosting, TLS, secret manager, network isolation, backups, health checks, and monitoring.
Evaluation coverage	Unit tests exist for route and output policy, but full clinical/user evaluation is limited.	Route test matrix, Bengali comprehension testing, accessibility testing, adverse-case review, and pilot governance.

6. Deployment and future controlled scope

Zone	Current capstone	Future controlled-pilot requirement
Frontend	React PWA runs locally; GitHub Pages publishes wireframe documentation only.	Production HTTPS hosting, accessibility monitoring, and release controls.
Backend	Spring Boot API runs locally outside Docker for the demo.	Managed deployment, secret manager, authentication, monitoring, and scale planning.
Data and AI	Docker Compose runs PostgreSQL and local Ollama. Ollama is not directly exposed to the browser.	Managed PostgreSQL, content lifecycle, network controls, backup/restore, model evaluation, and audit controls.
Extended automation	Agent, Skills, MCP gateway, service administration workflow, and persistent audit store are not implemented.	Allowlisted tools, least privilege, human approval, testing, no autonomous clinical or user-facing action.